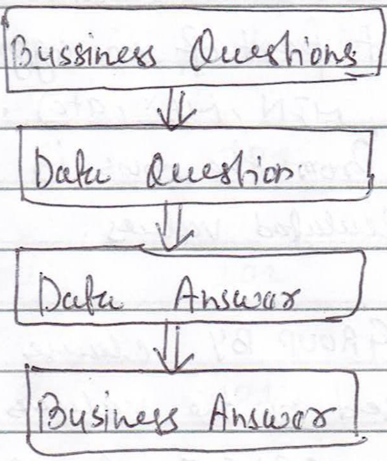
Lession 10 Building SQL Datasets for Analytical Reporting.

data base: analytical_reporting



Thinking through Analytical Dataset Requirements.

The core concept here is designing reusable analytical datasets that can be used to answer multiple questions and build various reports. This involves

1. Understanding Requirement
-) Anticipating the types of question that might be asked and dimension (e.g. time, vendor, product) and measures (e.g. sales, quantity) needed to answer them.

2. Choosing Granularity
-) Deciding the level of detail (e.g. daily, weekly) at which the dataset should be summarized.

3. Combining Data
-) Joining relevant tables to include all necessary fields (e.g. vendor_names, market_days) for reporting.

4) Summarizing data
Aggregating data (e.g. summing sales) to create a dataset that is easy to query and analyse.

Error Code 1055

→ In SQL, non-aggregated columns are columns in the SELECT clause that are not part of an aggregate function (e.g. SUM, COUNT, AVG, MIN, MAX, etc). These columns represent individual values from the rows in table, rather than summarized or calculated values.

→ When you use a GROUP BY clause in a query, you are grouping rows based on the values in one or more columns. Any column in the SELECT clause that is not part of an aggregate function must be included in the GROUP BY clause. This ensures that the query returns consistent and predictable results.

Aggregated Columns	Non-Aggregated Columns
(1) Column that use an aggregate function to summarize or calculate values across multiple rows	(1) Columns that represent individual values from the rows in the table. These must be included in the GROUP BY clause if used in the SELECT clause.
(2) SUM (quantity), AVG (price), count(*)	(2) market_date, vendor_id, product_name

Customer_purchases:

market-date	vandor-id	product-id	customer-id	quantity	Cost-to-customer-per-qty
2023-10-01	1	101	12	2.00	5.00
2023-10-01	1	102	15	3.00	3.50
2023-10-01	2	201	18	1.00	10.00
2023-10-02	1	101	12	4.00	5.00
2023-10-02	2	201	15	2.00	10.00
Date	Int	Int	Int	Decimal (10,2)	Decimal (10,2)

market-date-info

market_date	market_day	market_week	market_year	raining
2023-10-01	Sunday	40	2023	0
2023-10-02	Monday	40	2023	1
Date	Varchar(10)	Int	Int	Boolean

vandor

vandor_id	vandor_name	vandor_type
1	Marco's Peppers	Produce
2	Fresh Daily	Dairy
Int	Varchar(50)	Varchar(50)

PAGE NO.

Design the Reusable Data set - 1

```
CF Select
    cp.market-date,
    md.market-day,
    md.market-week,
    md.market-year,
    cp.vendor-id,
    v.vendor-name,
    v.vendor-type,
Round(sum(cp.quantity * cp.cost-to-customer
          -per-qty), 2) As sales.
From
    customer-purchases As cp.
    Left Join market-date-info As md.
        ON cp.market-date = md.market-date
    Left Join vendor As V
        ON cp.vendor-id = v.vendor-id.
Group BY
    cp.market-date, md.market-day,
    md.market-week, md.market-year,
    cp.vendor-id, v.vendor-name, v.vendor-type
Order by
    cp.market-date, cp.vendor-id;
```

market-date	market-day	market-week	market-year	vendor-id cp	vendor-name v	vendor-type v	sales cp
2023-10-01	Sunday	40	2023	1	Marco's Peppers	Produce	20.30
2023-10-01	Sunday	40	2023	2	Fresh Dairy	Dairy	10.00
2023-10-02	Monday	40	2023	1	Marco's Peppers	Produce	20.00
2023-10-02	Monday	40	2023	2	Fresh Dairy	Dairy	20.00

-- Use the Dataset for Analysis.

1. Total Sales Last Week.

```
Select
    SUM (sales) As total_sales_last_week
From
    (
    Select
        cp. market_date,
        Round(SUM (cp. quantity * cp. cost_to_customer_per_
                                qty), 2) As sales
    From
        customer_purchase As cp
    Where
        cp. market_date Between '2023-10-01' AND '2023-10-07'
    Group by
        cp. market_date
    ) As weekly_sales.
```

Subquery

Market_date	Sales
2023-10-01	3x5 + 3x3.5 + 1x10 = 30.5.
2023-10-02	4x5 + 2x10 = 40

outerquery

total_sales_last_week
30.5 + 40 = 70.50.

output

total_sales_lastweek
70.50

2. Sales by Vandor Type.

```
Salact
      v. vandor_type,
      SUM (cp. quantity * cp. cust-to-customer-per-qty) As
                                                  total-sales
from
      customar-purchases  As cp
      Left-Join vandor ~~as~~ v
            on cp. vandor-id = v. vandor id.
      Group by
            v. vandor_type;
```

Explaination

vandor_type	total. sales.
Marco's Papper	5X2+3X3+4X5 = 110+103+20 = 40.5
Fresh Daily	1X10+2X10 = 20+10 = 30.

	Vandor-type	total_sales.
▶	Marco's Pappar	40.6.
	Fresh Daily	30.

3) Sales on Raining vs. Non - Raining Days.

```
Select
    md.raining,
    SUM (cp.quantity * cp.cost_to_customer_per_qty) As
                                              total_sales.
From
    customer_purchase As cp.
    Left Join market_date_info As md
            ON cp.market_date = md.market_date.
    Group by
        md.raining;
```

Explaination

raining	total-sales.
0	5X2+3X3.5 + 1X10 = 30.5.
1	5X4+2X10 = 20+20 = 40

raining	total-sales.
0	30.5
1	40.

Using Custom Analytical Datasets in SQL
: CTEs and Views.

customer_purchase1

market_date	vendor_id	product_id	customer_id	quantity	cost_to_customer_per_qty
2023-10-01	1	101	12	2.00	5.00
2023-10-01	1	102	15	3.00	3.50
2023-10-01	2	201	18	1.00	10.50.
2023-10-02	1	101	12	4.00	5.00
2023-10-02	2	201	15	2.00	10.00.
Date	Int	Int	Int	Decimal (10,2)	Decimal (10,2).

market_date_info1

market_date	market_day	market_week	market_year	raining.
2023-10-01	Sunday	40	2023	0
2023-10-02	Monday	40	2023	1

vendor1

vendor_id	vendor_name	vendor_type.
1	Marco's Peppers	Produce
2	Fresh Dairy	Dairy.

dhawa dorje ghising

1) Using a CTE

```
with   sales_by_day_vandor As (
    Select
        cp.market_date,
        md.market_day,
        md.market_week,
        md.market_year,
        cp.vandor_id,
        v.vandor_name,
        v.vandor_type,
        Round (SUM ( cp.quantity * cp.cost_to_customer_per_
                                        qty), 2) As sales.
    From
        customer_purchase1 As cp.
        Left Join  market_date_info1 As md
                ON  cp.market_date = md.market_date.
        Left Join  vandor1   As v
                ON  cp.vandor_id = v.vandor_id.
    Group by
        cp.market_date, cp.vandor_id, md.market_day, md.
                market_week, md.market_year,
                        v.vandor_name, v.vandor_type
) we can't put(;) because CTE exists only during
    query execution
Select
    market_year,
    market_week,
    SUM (sales) As weekly_sales.
From
    sales_by_day_vandor
Group by
    market_year, market_week.
Order by
    market_year, market_week;
```

output

```
market_year     market_week     weekly_sales.
2023               40               70.50.
```

2) Views

- A view is a stored query that acts like a virtual table.
- Unlike CTEs, views are stored in the databases and can be reused across multiple queries.
- Views dynamically generate results based on the underlying tables each time they are referenced.

Differences

	CTE	View
storage	Temporary: exists only during query execution	Permanent: stored in the database
Reusability	Can only be reused within the same query	Can be reused across multiple queries.
Performance	slightly faster for one-time use	slightly slower due to dynamic generation.
use case	ideal for complex, one-time queries	ideal for reusable datasets and reporting.

Syntax

CTE:

```
With cte-name As (
select --
from --
Groupby --
)
select --
```

View:

```
Create View view-name As
select --
from --
Groupby --;
```

```
-- Create a view

Create View vw_sales_by_day_vendor1 As
  Select
     cp.market_date,
     md.market_day,
     md.market_week,
     md.market_year,
     cp.vendor_id,
      v.vendor_name,
      v.vendor_type
      Round (sum (cp.quantity * cp.cost_to_customer
                                        _per_qty),2) As sales
 From
     customer_purchases1 As cp
     Left Join market_date_info1 As md
             ON cp.market_date = md.market_date
     Left Join vendor1 As v
             ON cp.vendor_id = v.vendor_id
Group by
   cp.market_date, ~~md~~ md.market_day, md.market_week,
   md.market_year, cp.vendor_id, v.vendor_name,
   v.vendor_type; (Here ';' means permanently stored
                          in the database),

-- Query the view.
  Select
       market_year,
       market_week,
     SUM (sales) As weekly_sales,
  From
       vw_sales_by_day_vendor1
  Group
     market_year, market_week
```

Order by

market_year, market_week;

market_year	market_week	weekly_sales
▶ 2023	40	70.50

drawa dorje ghising

Taking SQL Reporting Further.

welle do

1) creating Views.
2) Adding Calculated fields.
3) Using Window functions.
4) Answering Bussiness Qustion.

vendor-inventory2

market-date	vendor-id	product-id	quantity	Original-price
2023-10-01	1	101	50	5.00
2023-10-01.	1	102	30	3.50
2023-10-01	2	201	20	10.00
2023-10-02	1	101	40	5.00
2023-10-02	2	201	15	10.00
Date	Int	Int	Int	Decimal (10,2)

Customer-purchases2

market-date	vendor-id	product-id	Customer -id	quantity	cost-to-customer-per-qty
2023-10-01	1	101	12	2.00	6.00
2023-10-01	1	102	15	3.00	4.00
2023-10-01	2	201	18	1.00	12.00
2023-10-02	1	101	12	4.00	6.00
2023-10-02	2	201	15	2.00	12.00
Date	Int	Int	Int	Decimal (10,2)	Decimal (10,2)

vendor2

vendor-id	vendor-name	vendor-type
1	Marcols Peppers	Produce
2	Fresh Dairy	Dairy
Int	Varchar(50)	Varchar(50)

product2

product-id	product-name
101	Banana Peppers
102	Jalapenos
201	cheese
Int	Varchar(50)

-- Create a Reusable View.

```
Create View   vw-sales-per-date-vendor-product As
Salect
     vi. market -date,
     vi. vendor-id,
      v. vendor -name,
     vi. product-id,
      p. product-name,
      vi. quantity as  quantity-available
     Coalesce (sales.quantity -sold, 0) As quantity-sold,
     Round ((Coalesce (sales. quantity-sold, 0) / vi. quantity)
                         * 100, 2) As percent-of-available-sold,
      vi. original-price,
     (vi. original-price * Coalesce (sales quantity-sold, 0)) -
           Coalesce (sales. total-sales, 0) As discount-amount,
      Coalesce (sales. total-sales, 0) As total-sales.
  From
       vendor-inventory2 As vi
       Left Join (
             Select
                  market-date,
                  vendor-id,
                  product-id,
                  SUM (quantity) As quantity-sold.
                  SUM (quantity* cust-to-customer-per-qty) As total-sales
       From
            customer-purchases2
       Group by
               market date, vander-id, product-id.
      ) As sales.
  ON   vi. market-date = sales. market-date.
  AND  vi. vendor_id  = sales vendor_id
  AND  vi. product_id = sales. product-id
   Left Join  vendor2 As v
```

```
on vi. vandor_id = v. vandor_id.
Left Join product2 As p
   On vi. product_id = p. product_id.

Order by
   vi. market.date, vi.vendor-id, vi. product-id;
```

Sub-query Joining.

market-date	vandor-id	product-id	quantity-sold	total-sales.
2023-10-01	1	101	2.00	12.
2023-10-01	1	102	3.00	12.
2023-10-01	2	201	1.00	12.
2023-10-02	1	101	4.00	24
2023-10-02	2	201	2.00	24.

dhawa dorje dhising

output

▶ market_date(vi)	vendor-id(vi)	vendor_name(v)	product-id (vi)	product_name (p)	quantity_available (vp)	quantity_sold (sales)	% of available_sold	original_price	discount_amount	total_sales (sales)
2023-10-01	1	Marco's Peppers	101	Banana Peppers	50	2.00	4.00	5.00	-2	12
2023-10-01	1	Marco's Peppers	102	Jalapenos	30	3.00	10.00	3.50	-1.5	12
2023-10-01	2	Fresh Daily	201	Cheese	20	1.00	5.00	10.00	-2	12
2023-10-02	1	Marco's Peppers	101	Banana peppers	40	4.00	10.00	5.00	-4	24
2023-10-02	2	Fresh Daily	201	Cheese	15	2.00	13.33	10.00	-4	24

view_table_name = vw_sales_per_date_vendor_product

select * from vw_sales_per_date_vendor_product;

```
-- Query the view.

-- Example 1: percentage of Sales per Vandor per Market
                 Date Sql.

Select
      market_date,
      vandor_name,
      product_name
      quantity_available,
      quantity_sold,
      percent_of_available_sold,
      total_sales,
      Round ((total_sals / SUM (total_sales) Over( Partition by
              market_date, vandor_id)) * 100,2) As
                              percent_of_vandor_sales.
   From
        vw_sales_per_date_vandor_product
   Order by
        market_date, vandor_id, product_id;
```

Aspect	Partition By	vs	Group By
How collapsing	Does not collapse rows; retains all rows		Collapses rows into one row per group.
Aggregate functions	Can use aggregate functions without collapsing rows		Must use aggregate functions to summarize groups.
output	Retains all rows: adds calculated columns		Returns one row per group.
use Case	Used for calculations within groups (e.g. percentage)		used for summarizing data (e.g. totals, averages)

dhawa dorje ghising

PAGE NO.
DATE / /

Role of Over in window Functions

-> Defines the window or partition over which the calculation is performed.

-> allow you to perform calculations within groups without collapsing the rows.

-> Enables advanced analytics like running totals, rankings and percentages.

Market-date	vendor name	product name	quanty-available	quanty-sold	percent-of-available-sold	total-sales	Percent-of-vendor-sales
2023-10-01	MarcosPepper	Banana Peppers	50	2.00	4.00	12.0000	50.00
2023-10-01	Marco's Peppers	Jalapenos	30	3.00	10.00	12.0000	50.00
2023-10-01	Fresh Dairy	Cheese	20	1.00	5.00	12.0000	100.00
2023-10-02	Marco's Peppers	Banana Peppers	40	4.00	10.00	24.0000	100.00
2023-10-02	Fresh Dairy	Cheese	15	2.00	13.00	24.0000	100.00

output

dhawa dorje ghising

```
-- Example 2 : sales for a Specific Vendor and Date
               Range

Select
    market_date,
    vendor_name,
    product_name,
    quantity_available,
    quantity_sold,
    total_sales
From
    vw_sales_per_date_vendor_product.
Where
    market date Between '2023-10-01' AND '2023-10-02'
      And  vendor_name = 'Marco's Peppers'
Order by
      market_date, product_id;
```

<u>output</u>

```
                                            quantity    quantity    total
market_date  vendor_name      product_name   _available  _sold       _sales.

2023-10-01   Marco's Peppers  Banana Peppers   50         2.00        12.0000
2023-10-01   Marco's Peppers  Jalapenos        30         3.00        12.0000
2023-10-02   Marco's Peppers  Banana Peppers   40         4.00        24.0000
```

dhawa dorje ghising